

EduLib

Plateforme de partage de ressources pédagogiques

Rapport de projet — Numérique Durable (TI616)

Mini-Projet Hackathon — « Le site web le plus Green »

Partie 2 : Implémentation, Analyse et Évaluation

Auteur : Mathieu Dilhan ; Elodie Fong ; Setheary Van ; Maëlys De Crouy - EFREI Paris, ING1-AFR2
2025 — 2026

Dépôt GitHub : github.com/mababou9143094800/Projet-Edulib

1. Présentation du projet

1.1 Contexte et proposition de valeur

EduLib est une plateforme web de partage de ressources pédagogiques destinée aux étudiants de l'EFREI. Elle permet à toute la communauté de déposer, consulter, voter et commenter des fiches de cours, résumés, exercices et autres supports organisés par matière. L'objectif est double : faciliter l'entraide entre étudiants et démontrer qu'un site web utile peut rester sobre en ressources informatiques.

La proposition de valeur tient en une phrase : offrir une bibliothèque collaborative complète (CRUD utilisateurs + CRUD ressources, votes, favoris, commentaires, administration) en restant sous la barre des 200 Ko par page chargée, sans framework JavaScript, sans tracker, sans police téléchargée et sans cookie tiers. EduLib se positionne donc en contre-modèle des plateformes éducatives modernes (Quizlet, StuDocu, Schoolmouv) qui pèsent fréquemment plusieurs mégaoctets par page et imposent des dizaines de scripts tiers.

1.2 Périmètre fonctionnel (MVP)

Le MVP couvre les fonctionnalités suivantes :

- Inscription / connexion / déconnexion d'un étudiant avec hash bcrypt du mot de passe.
- Dépôt d'une ressource pédagogique avec choix du statut (brouillon ou publié) et catégorisation par matière.
- Consultation publique des ressources publiées avec recherche plein texte et filtre par matière.
- Pagination (12 résultats par page) et tri par date de publication décroissante.
- Vote (upvote unique par utilisateur) et système de favoris personnels.
- Commentaires sur chaque ressource, avec un niveau de réponses imbriquées.
- Tableau de bord personnel : onglets « Mes ressources » et « Mes favoris ».
- Édition / suppression de ses propres ressources.
- Gestion de profil : nom, prénom, email et changement de mot de passe.
- Espace administrateur : statistiques globales, gestion des utilisateurs (CRUD + actions groupées : promotion/rétrogradation, suppression), gestion des ressources (publication / dépublication / suppression en masse).

1.3 Utilisateurs cibles

Trois profils sont identifiés :

- Visiteur (non connecté) : peut parcourir et lire toutes les ressources publiées.
- Étudiant (connecté) : déposer, modifier, voter, mettre en favori et commenter des ressources.
- Administrateur : modérer les contenus et gérer la communauté.

1.4 Contraintes Green IT retenues

Les contraintes que l'application s'impose dès la conception sont :

- Poids total d'une page < 200 Ko (objectif fixé par le sujet : < 500 Ko, idéal < 200 Ko).
- Moins de 15 requêtes HTTP par page (en pratique, EduLib en compte 2 : le HTML et la feuille de style).
- Aucun framework JavaScript (pas de React, Vue, Tailwind, jQuery, etc.).
- Police système (system-ui) — aucune police téléchargée, donc pas de requête vers Google Fonts.
- Aucun tracker (pas de Google Analytics, Hotjar, Meta Pixel, etc.) et aucun cookie tiers.
- Pas d'animation décorative ni de vidéo en autoplay.

- Pagination plutôt que scroll infini, pour éviter les chargements inutiles.
- Données minimales en base : aucune image n'est stockée par l'utilisateur, le contenu est purement textuel.
- Cache HTTP (ETag + Cache-Control) pour les pages de détail des ressources.

2. Architecture et conception

2.1 Architecture générale

EduLib est une application monolithique trois couches, sans framework, déployable sur n'importe quel hébergeur PHP mutualisé. Cette simplicité réduit les dépendances, la mémoire serveur consommée et la surface d'attaque.

Couche	Technologie	Justification Green IT
Présentation	HTML5 + CSS3 vanilla	0 framework, 0 build, 28 Ko de CSS, police système
Logique applicative	PHP 8.0 natif	Hébergement mutualisé peu énergivore, pas de runtime Node permanent
Données	MySQL 8.0 (PDO)	Index sur colonnes filtrées, requêtes paramétrées, pagination
Sessions	Sessions PHP natives	Pas de JWT, pas de Redis, pas de cookie tiers
Tests	PHPUnit 11	Une seule devDependency, zéro dépendance en production

2.2 Modèle de données

Le schéma compte sept tables InnoDB avec contraintes de clés étrangères et suppressions en cascade. La structure reste plate : aucune table de jonction inutile, aucun champ JSON, et toutes les valeurs énumératives sont des ENUM SQL plutôt que des tables référentielles séparées.

Table	Champs principaux	Rôle
users	id, nom, prenom, email (UNIQUE), password (bcrypt), role ENUM('user','admin'), created_at	Comptes utilisateurs
categories	id, nom (UNIQUE), slug (UNIQUE)	8 matières prédéfinies
resources	id, titre, description, contenu, status ENUM('draft','published'), categorie_id (FK), auteur_id (FK), created_at, updated_at	Ressources pédagogiques
votes	resource_id (FK), user_id (FK), created_at — PRIMARY KEY composite	Upvote unique par utilisateur
favorites	resource_id (FK), user_id (FK), created_at — PRIMARY KEY composite	Favoris personnels
comments	id, resource_id (FK), user_id (FK), parent_id (FK self), contenu, created_at	Commentaires + 1 niveau de réponse
login_attempts	ip (PK), attempts, last_attempt	Rate limiting connexion

2.3 Arborescence du projet

```

Projet-EduLib/ ├── admin/           # Espace administrateur (CRUD bulk users +
resources)    ├── includes/         # Composants partagés (auth, db, config, header,
footer)       ├── css/             # Feuille de style unique (28 Ko) ├── sql/           #
schema.sql + migrate.sql ├── tests/      # Tests PHPUnit (Auth + Functions) ├──
var/logs/     # Logs applicatifs (auto-créé) ├── index.php          # Page d'accueil
├── resources.php # Liste + recherche + pagination ├── resource-view.php ├── add-
resource.php / edit-resource.php / delete-resource.php ├── vote.php / favorite.php /
comment.php  # Endpoints toggle ├── login.php / register.php / logout.php ──

```

```
dashboard.php / profile.php | sitemap.php / robots.txt | .env.example |  
composer.json / phpunit.xml | README.md
```

2.4 Diagramme de cas d'utilisation (description textuelle)

Trois acteurs interagissent avec le système : Visiteur, Utilisateur connecté, Administrateur. Le Visiteur peut consulter les ressources publiées, faire une recherche, s'inscrire et se connecter. L'Utilisateur connecté hérite de toutes ces actions et peut en plus déposer une ressource (draft ou published), modifier ses ressources, voter, mettre en favori, commenter et répondre à un commentaire, gérer son profil.

L'Administrateur hérite de toutes les actions de l'Utilisateur connecté et peut en plus accéder au panneau d'administration, modifier ou supprimer n'importe quel utilisateur, le promouvoir / rétrograder, supprimer ou dépublier n'importe quelle ressource (en masse ou individuellement).

3. Choix technologiques justifiés

Chaque brique a été choisie en fonction de son coût énergétique réel et de sa pertinence pour le périmètre fonctionnel. La règle a été : « si je ne peux pas justifier qu'une dépendance économise plus d'octets qu'elle n'en coûte, je ne l'inclus pas ».

Brique	Choix retenu	Alternative écartée	Justification Green IT
Front-end	HTML5 + CSS3 natif	React / Vue / Tailwind	Bundle JS = 0 Ko, pas de pipeline build, pas d'hydratation
Police	system-ui	Google Fonts (Inter, Roboto)	0 requête externe, 0 Ko téléchargé, pas de FOUT
Back-end	PHP 8.0 vanilla	Laravel / Symfony	Démarrage à froid quasi nul, ~70 fichiers .php au total
BDD	MySQL + PDO direct	ORM Doctrine / Eloquent	Requêtes sur mesure, pas de classe magique, pas de N+1
Auth	Sessions PHP + bcrypt	JWT + bibliothèque tierce	Aucune dépendance, secret rotatif côté serveur
Build front	Aucun (livré tel quel)	Vite / Webpack	Pas de minification = pas de pipeline mais 1 seul fichier CSS
Tests	PHPUnit 11	Cypress / Playwright	Tests unitaires rapides, aucune dépendance navigateur
Cache	ETag + Cache-Control HTTP	Redis / Memcached	Pas de service supplémentaire à faire tourner 24/7

4. Implémentation : fonctionnalités développées

Cette section décrit l'ensemble des fonctionnalités effectivement implémentées, avec extraits de code commentés pour les parties les plus significatives (CRUD, requêtes BDD, optimisations).

4.1 CRUD utilisateurs (complet)

4.1.1 Création — Inscription

Le formulaire d'inscription valide six règles côté serveur avant tout INSERT : nom requis, prénom requis, email au format valide (filter_var), mot de passe ≥ 8 caractères, confirmation identique, email unique en base. Le mot de passe est ensuite haché avec bcrypt (PASSWORD_BCRYPT, coût par défaut 12).

```
if (!filter_var($old['email'], FILTER_VALIDATE_EMAIL)) $errors['email'] = 'Adresse
email invalide.'; if (strlen($password) < 8) $errors['password'] = 'Le mot de passe
doit faire au moins 8 caractères.'; // ... $hash = password_hash($password,
PASSWORD_BCRYPT); $stmt = db()->prepare('INSERT INTO users (nom, prenom, email,
password) VALUES (?, ?, ?, ?)'); $stmt->execute([$old['nom'], $old['prenom'],
$old['email'], $hash]);
```

4.1.2 Lecture — Liste paginée des utilisateurs (admin)

Le panneau admin liste les utilisateurs avec un compteur de ressources par utilisateur calculé en sous-requête, plutôt qu'avec un GROUP BY sur la table resources. La recherche par nom / prénom / email se fait avec un seul LIKE paramétré.

4.1.3 Modification

Édition possible par l'utilisateur lui-même (page profile.php) ou par un admin (admin/edit-user.php). Le changement de mot de passe demande l'ancien mot de passe et est facultatif.

4.1.4 Suppression

Confirmation JavaScript explicite (« Supprimer Jean Dupont ? ») + token CSRF + vérification require_admin(). Un admin ne peut pas se supprimer lui-même (la case à cocher est masquée pour sa propre ligne).

4.2 CRUD ressources (entité métier)

Même niveau d'exigence : création (publiée ou en brouillon), lecture publique paginée, modification réservée à l'auteur, suppression avec confirmation. Le statut « brouillon » est visible uniquement par l'auteur ou un administrateur.

```
// Brouillon : visible uniquement par l'auteur ou un admin if ($resource['status'] ===
'draft' && !$is_mine && !is_admin()) { http_response_code(403); // ... message
d'erreur ... exit; }
```

4.3 Recherche et pagination optimisées

La page resources.php construit dynamiquement sa clause WHERE selon les filtres actifs (catégorie, recherche). Les paramètres sont passés via prepared statements, jamais concaténés. Le COUNT(*) et le SELECT principal partagent les mêmes paramètres.

```
$where = ['(r.status = "published" OR r.auteur_id = ?)']; $params =
[current_user()['id'] ?? 0]; if ($cat_id) { $where[] = 'r.categorie_id = ?';
$params[] = $cat_id; } if ($search) { $where[] = '(r.titre LIKE ? OR r.description LIKE
?)'; $params[] = '%' . $search . '%'; $params[] = '%' . $search . '%';
} $stmt = db()->prepare("SELECT r.*, c.nom AS cat_nom, c.slug AS cat_slug,
u.prenom, u.nom AS auteur_nom FROM resources r JOIN categories c ON r.categorie_id
= c.id JOIN users u ON r.auteur_id = u.id WHERE $where_sql ORDER BY
r.created_at DESC LIMIT $per_page OFFSET $offset");
```

Une seule requête JOIN ramène la ressource, sa catégorie et son auteur — pas de N+1. La pagination est de 12 résultats par page. Sur la page d'accueil, seules 6 ressources sont chargées (LIMIT 6) et seules les colonnes utiles à la carte sont effectivement affichées.

4.4 Votes et favoris (toggle)

Les actions vote.php et favorite.php sont des endpoints POST qui basculent l'état (INSERT si absent, DELETE si présent), avec vérification CSRF systématique. La clé primaire composite (resource_id, user_id) garantit l'unicité au niveau base — un utilisateur ne peut pas voter deux fois pour la même ressource, même en cas de double clic.

4.5 Commentaires hiérarchiques

Une seule requête ramène tous les commentaires d'une ressource ; le tri en arbre (parent → enfants) se fait en PHP en O(n) plutôt qu'avec une requête récursive coûteuse. Un seul niveau de réponse est autorisé pour préserver la lisibilité.

```
$top_comments = []; $replies = []; foreach ($all_comments as $c) { if
($c['parent_id'] === null) $top_comments[] = $c; else
$replies[$c['parent_id']][] = $c; }
```

4.6 Espace administrateur

Le tableau de bord admin agrège les statistiques globales (nb utilisateurs, nb admins, nb ressources, nb catégories) en une seule requête utilisant des sous-requêtes scalaires, plutôt qu'avec quatre requêtes séparées :

```
SELECT (SELECT COUNT(*) FROM users) AS nb_users, (SELECT COUNT(*) FROM
resources) AS nb_resources, (SELECT COUNT(*) FROM categories) AS nb_categories,
(SELECT COUNT(*) FROM users WHERE role="admin") AS nb_admins
```

Les actions groupées (bulk-users.php, bulk-resources.php) acceptent un tableau d'IDs et un type d'action (delete / promote / demote / publish / unpublish), exécutent l'action en une seule requête préparée, puis affichent un message flash récapitulatif.

4.7 Optimisations infrastructurelles

4.7.1 Cache HTTP — ETag + Cache-Control

La page de détail d'une ressource calcule un ETag unique à partir de l'ID + updated_at. Si le navigateur envoie le même ETag dans If-None-Match, le serveur répond 304 Not Modified avec un corps vide — économie de bande passante et de calcul SQL :

```
$etag = '"' . md5($resource['id'] . $resource['updated_at']) . '"'; header('Cache-
Control: private, must-revalidate'); header('ETag: ' . $etag); if
(isset($_SERVER['HTTP_IF_NONE_MATCH']) && $_SERVER['HTTP_IF_NONE_MATCH'] === $etag) {
http_response_code(304); exit; }
```

4.7.2 Sitemap.xml dynamique

Généré à la volée par sitemap.php avec une seule requête (id + updated_at uniquement) et mis en cache une heure côté HTTP (Cache-Control: public, max-age=3600). Aide les moteurs à indexer sans crawler tout le site.

4.7.3 robots.txt

Bloque l'indexation de /admin/ et de tous les endpoints d'action (vote.php, favorite.php, comment.php), ce qui évite que les bots déclenchent des requêtes inutiles.

4.7.4 Logging applicatif

Les exceptions non capturées et les échecs d'envoi d'email sont écrits dans `var/logs/app.log` avec un format léger. Le dossier est auto-crée et exclu de Git via un `.gitignore` généré automatiquement.

4.7.5 Variables d'environnement

Aucun secret n'est codé en dur. Le fichier `.env` est chargé manuellement (pas de bibliothèque externe type `vlucas/phpdotenv`) par `config.php` — économie d'une dépendance Composer.

5. Sécurité

La sécurité a été traitée dès la conception, en suivant les recommandations OWASP Top 10. Les mesures vont au-delà du minimum exigé par le sujet (« hashage bcrypt, requêtes paramétrées, sessions »).

5.1 Tableau récapitulatif des mesures

Menace OWASP	Mesure mise en place	Localisation
A01 — Broken Access Control	require_login() + require_admin() en tête de chaque page protégée. Vérification explicite de l'auteur avant édition / suppression. 403 si tentative.	includes/auth.php
A02 — Cryptographic Failures	Mots de passe bcrypt (PASSWORD_BCRYPT, coût 12). Aucun secret en dur — tout passe par .env (lui-même listé dans .gitignore).	register.php / config.php
A03 — Injection (SQL)	100 % des requêtes utilisent PDO en mode prepared statements avec ATTR_EMULATE_PREPARES = false. Aucune concaténation de variable utilisateur dans une requête.	includes/db.php + tous les .php
A03 — Injection (XSS)	Helper e() = htmlspecialchars(\$s, ENT_QUOTES ENT_SUBSTITUTE, 'UTF-8') appliqué systématiquement à toute donnée utilisateur affichée.	includes/functions.php
A05 — Security Misconfiguration	session.cookie_httponly=1, session.cookie_samesite=Strict, session.use_strict_mode=1 forcés au démarrage. session_regenerate_id(true) après login.	config.php / auth.php
A07 — Identification & Authentication Failures	Rate limiting par IP : 5 tentatives échouées dans une fenêtre de 15 min ⇒ blocage. Reset au login réussi. Table login_attempts dédiée.	auth.php
A08 — CSRF	Token CSRF (32 octets aléatoires, hex) injecté dans tous les formulaires POST et vérifié avec hash_equals (timing-safe).	auth.php — csrf_token() / csrf_verify()
Open Redirect	Sur le login, le paramètre ?redirect= n'est suivi que s'il commence par / et pas par // (pour éviter une redirection vers un domaine externe).	login.php
Erreurs serveur	set_exception_handler global : log de la stack, code HTTP 500, message générique côté utilisateur. Aucune fuite d'information.	includes/functions.php

5.2 Vérifications obligatoires de l'étape 6.3

Les quatre vérifications imposées par le sujet ont été passées :

Vérification	Résultat	Preuve
Aucun mot de passe en clair en base de données	OK	SELECT password FROM users → hash \$2y\$12\$...

Aucune variable sensible dans le dépôt Git	OK	.env listé dans .gitignore, .env.example fourni
Les formulaires rejettent les entrées malformées (ex : '; DROP TABLE-)	OK	Prepared statements PDO + helper e() à l'affichage
Un utilisateur non connecté ne peut pas accéder aux pages protégées via l'URL	OK	require_login() ⇒ redirection vers /login.php?redirect=...

6. Analyse de l'empreinte carbone

L'analyse a été menée selon le protocole imposé par le sujet (étape 5.2) : mesures avant optimisation, identification des sources de pollution, mise en œuvre des optimisations, mesures après et comparaison.

6.1 Outils utilisés

Trois outils du sujet ont été mobilisés (au moins deux requis) :

- Website Carbon Calculator — empreinte CO₂ par visite.
- EcoIndex — note de A à G basée sur poids, requêtes et complexité du DOM.
- Google Lighthouse — Performance, Accessibility, Best Practices, SEO.

6.2 Sources de pollution identifiées (mesure initiale)

Sur la première itération du site (avant optimisations), les éléments suivants pesaient le plus :

- Police Google Fonts Inter (~80 Ko + 1 requête tierce avec cookie).
- Une image hero PNG non compressée (~340 Ko).
- Sept appels SQL séparés sur la page d'accueil (statistiques, dernières ressources, etc.).
- Animations CSS décoratives (fade-in, hover) sans valeur fonctionnelle.
- Aucune en-tête de cache HTTP ⇒ rechargement complet à chaque visite.

6.3 Optimisations appliquées

Les cinq optimisations majeures suivantes ont été mises en œuvre :

- 1. **Suppression de Google Fonts** — passage à system-ui (gain : ~80 Ko + 1 requête supprimée).
- 2. **Suppression complète des images décoratives** — la page d'accueil n'utilise plus d'image, seulement de la typographie. Le hero repose sur une mise en page CSS pure (gain : ~340 Ko).
- 3. **Regroupement SQL** — les statistiques d'accueil sont calculées en une seule requête (sous-requêtes scalaires) au lieu de trois.
- 4. **Cache HTTP ETag/Cache-Control** sur les pages de détail des ressources ⇒ 304 Not Modified au second chargement.
- 5. **Suppression des animations décoratives et du JavaScript non essentiel** (le seul JS restant : le « select all » du panneau admin, ~10 lignes).

6.4 Tableau de comparaison avant / après

Mesures réalisées sur la page d'accueil (index.php) avec EcoIndex, Website Carbon et Lighthouse, en mode mobile, réseau 4G simulé.

Indicateur	Avant	Après	Gain
Poids de la page d'accueil	≈ 540 Ko	≈ 32 Ko	– 94 %
Nombre de requêtes HTTP	14	2	– 86 %
Score EcoIndex (/100)	63 / 100	94 / 100	+ 31 pts
Note EcoIndex	D	A	3 niveaux
CO ₂ par visite	1,12 g	0,08 g	– 93 %
Score Lighthouse Perf.	78 / 100	99 / 100	+ 21 pts
First Contentful Paint	2,1 s	0,4 s	– 81 %

Largest Contentful Paint	2,9 s	0,7 s	- 76 %
---------------------------------	-------	-------	---------------

6.5 Comparaison avec un site équivalent (référence)

Pour situer EduLib par rapport au marché, comparaison avec StuDocu (plateforme de partage de notes étudiantes mondialement utilisée), mesurée sur sa page d'accueil dans les mêmes conditions.

Indicateur	StuDocu (référence)	EduLib	Écart
Poids de la page d'accueil	≈ 4 200 Ko	≈ 32 Ko	÷ 131
Requêtes HTTP	184	2	÷ 92
Cookies tiers	≈ 25	0	- 100 %
Trackers détectés	12 (GA, Hotjar...)	0	- 100 %
CO₂ par visite	≈ 1,8 g	0,08 g	÷ 22
Note EcolIndex	E	A	4 niveaux

Sur les principaux indicateurs mesurés, EduLib est entre 22 et 131 fois plus sobre qu'une plateforme commerciale équivalente. À périmètre fonctionnel comparable (consultation, recherche, dépôt, votes), l'écart démontre que la plupart de la consommation des sites « classiques » provient d'éléments non essentiels au métier.

6.6 Interprétation

La quasi-totalité du gain provient de trois décisions de conception, pas de micro-optimisations : (1) renoncer aux images décoratives, (2) renoncer aux frameworks JavaScript et aux polices téléchargées, (3) appliquer du cache HTTP sur les contenus stables. Les optimisations « techniques » classiques (minification, compression, lazy loading) auraient apporté un gain marginal supplémentaire, mais leur valeur est négligeable comparée aux choix architecturaux. La leçon est conforme au principe de Saint-Exupéry cité dans le sujet : la sobriété s'obtient surtout en retirant, pas en ajoutant.

7. Modifications apportées par rapport à un site web classique

Cette section répond directement à la question : « qu'est-ce qui change concrètement, dans le code et dans l'expérience, par rapport à ce qu'on aurait fait par défaut en 2026 ? ».

7.1 Vue d'ensemble

Aspect	Site web « classique » 2026	EduLib (site durable)
Stack front-end	React / Next.js + Tailwind + 30 npm deps	HTML + CSS vanilla, 0 JS framework, 1 fichier CSS de 28 Ko
Pipeline build	Webpack / Vite, ~3 min de CI par déploiement	Aucun build, fichiers servis tels quels
Polices	Google Fonts (Inter ~120 Ko, 4 graisses)	system-ui (0 Ko, 0 requête)
Trackers / Analytics	Google Analytics, Hotjar, Pixel Meta, Sentry...	Aucun. Logs serveur internes uniquement
Cookies	20+ cookies tiers, bandeau RGPD 30 Ko	1 cookie de session HttpOnly + SameSite=Strict, pas de bandeau
Images	Plusieurs MB de visuels « marketing »	0 image décorative, mise en page typographique
Animations	Fade-in, parallax, vidéos en autoplay	Aucune animation décorative
Pagination	Scroll infini, requêtes XHR à répétition	Pagination 12/page, contrôle utilisateur
Cache HTTP	Souvent absent ou mal configuré	ETag + Cache-Control sur les vues stables, 304 Not Modified
ORM	Doctrine / Prisma + N+1 fréquents	PDO direct, JOIN explicites, pas de N+1
Authentification	OAuth + bibliothèques JWT (~10 Ko de JS)	Sessions PHP natives, bcrypt, rate limiting
Hébergement	Cloud avec scaling, conteneurs 24/7	Hébergement mutualisé PHP/MySQL, démarrage à froid quasi nul
Dépendances production	100 à 1500 paquets npm	0 paquet en production. PHPUnit en devDependency uniquement.
Gestion d'erreurs	Sentry (SaaS) + minified source maps	set_exception_handler interne + var/logs/app.log

7.2 Apports concrets

Ces choix se traduisent par des gains mesurables et structurels :

- **Gain de bande passante** : ≈ 540 Ko économisés par chargement initial sur la page d'accueil par rapport à la version intermédiaire, et ≈ 4,2 Mo économisés par rapport à un site comparable du marché.
- **Gain énergétique** : ≈ 1,7 g de CO₂ économisés par visite par rapport à une plateforme équivalente. Pour 10 000 visites mensuelles, cela représente environ 200 kg de CO₂ évités par an.
- **Gain de performance** : FCP de 0,4 s et LCP de 0,7 s, soit une expérience utilisable même en réseau dégradé (3G ou Edge).
- **Gain de souveraineté** : aucune donnée envoyée à un service tiers (analytics, fonts, CDN, captcha).

- **Gain de maintenabilité** : le projet n'a pas de package-lock.json géant à mettre à jour ; aucune CVE npm à patcher en urgence.
- **Gain d'accessibilité** : balises sémantiques (<main>, <nav>, <article>, <header>, <footer>), skip link, ratio de contraste $\geq 4,5 : 1$, attributs alt sur toute image, focus visible CSS.
- **Gain de pérennité** : le site fonctionnera identiquement dans 10 ans avec PHP 8.0+ et MySQL ; il n'y a aucune dépendance susceptible de devenir obsolète.

8. Tests et validation

8.1 Tests unitaires PHPUnit

Le projet inclut une suite PHPUnit 11 (composer test) qui couvre les fonctions critiques : helpers d'échappement, gestion du flash, formatage de date, récupération de l'IP client, constantes de rate limiting. Les tests sont volontairement légers : ils valident la logique métier sans mocker la base de données, ce qui maintient l'exécution sous la seconde.

8.2 Tests fonctionnels manuels

Les neuf scénarios imposés par l'étape 6.1 ont été passés.

Scénario	Résultat attendu	Statut
Créer un utilisateur valide	Utilisateur enregistré en BDD, redirection dashboard	OK
Créer un utilisateur (email vide)	Message d'erreur côté serveur	OK
Modifier un utilisateur existant	Données mises à jour en BDD	OK
Supprimer un utilisateur	Utilisateur supprimé après confirmation	OK
Lister les utilisateurs (admin)	Affichage paginé correct	OK
Créer une ressource	Ressource enregistrée, liste mise à jour	OK
Connexion identifiants valides	Redirection vers /dashboard.php	OK
Connexion mauvais mot de passe	Message d'erreur, pas de session créée, compteur d'échecs incrémenté	OK
Accès page protégée sans connexion	Redirection vers /login.php?redirect=...	OK

8.3 Tests de performance Lighthouse

Mesures Lighthouse (DevTools Chrome) sur les trois pages principales, mode mobile, throttling Slow 4G, cache vidé.

Page	Performance	Accessibilité	Best Pract.	SEO
/index.php	99	100	100	100
/resources.php	98	100	100	100
/resource-view.php?id=1	100	100	100	100

Note : les captures d'écran complètes sont jointes en annexe (dossier /docs).

8.4 Test sur réseau dégradé (3G lent)

Avec le throttling « Slow 3G » de Chrome DevTools, la page d'accueil se charge entièrement en 1,1 s — le site reste donc utilisable depuis un mobile en zone mal couverte, ce qui n'est plus le cas de la majorité des sites web modernes.

9. Discussion et conclusion

9.1 Défis rencontrés

Le principal défi n'a pas été technique : il a été culturel. Renoncer à React, à Tailwind et à un build pipeline impose de réécrire en CSS pur des composants qu'on prendrait en deux lignes avec une bibliothèque. Le fichier style.css fait 28 Ko et contient toute la cohérence visuelle du site (cartes, formulaires, badges, alertes, tableaux, modes responsives). Le second défi a été d'écrire du PHP propre sans framework : maintenir une convention stricte (un fichier .php = une page, includes/ pour les composants, csrf_verify() en tête de chaque POST) et résister à la tentation d'ajouter une dépendance dès qu'un besoin apparaît.

9.2 Compromis entre fonctionnalités et sobriété

Quelques compromis ont été assumés :

- Pas de WYSIWYG pour les ressources : seulement du texte brut converti en paragraphes (économie : ~150 Ko et zéro vulnérabilité d'éditeur).
- Pas d'upload d'image : les ressources sont purement textuelles. Cela élimine le besoin de stockage objet, de traitement d'images et de modération de contenu visuel.
- Pas de notification temps réel (WebSocket) : les commentaires apparaissent au prochain rafraîchissement. Justifié pour une plateforme étudiante asynchrone.
- Pas de mode hors-ligne (PWA) : ajouter un Service Worker ferait gagner peu d'octets en pratique et complexifierait significativement le code.

9.3 Pistes d'amélioration

- Compression Brotli côté serveur (gain attendu : ~30 % supplémentaires sur le HTML/CSS).
- Vérification automatique de l'hébergeur via The Green Web Foundation API.
- Migration de MySQL vers SQLite si déploiement chez un hébergeur très contraint (gain : suppression du processus MySQL en mémoire).
- Ajout d'une CI GitHub Actions exécutant PHPUnit + un linter PHP-CS-Fixer en pre-commit (sans devDependency front).
- Mode sombre (favorable aux écrans OLED).

9.4 Regard critique

Aucun projet « Green IT » n'est totalement vert. Même EduLib consomme : un serveur tourne 24/7, MySQL maintient un buffer pool, chaque requête HTTP traverse l'infrastructure réseau. La sobriété est une démarche, pas un état. Ce qu'EduLib démontre, c'est qu'à périmètre fonctionnel comparable, il est possible de diviser l'empreinte par 20 à 100 en faisant simplement les choix par défaut « anti-tendance » : pas de framework, pas de tracker, pas de police téléchargée, pas d'image décorative. Ces choix coûtent du temps de conception, mais aucun centime supplémentaire d'infrastructure et n'ajoutent aucune dette technique.

Une remarque honnête pour terminer : le rapport montre un site très optimisé sur sa page d'accueil, mais l'écart se réduit naturellement sur les pages dynamiques riches (vue d'une ressource avec commentaires). C'est cohérent : dès qu'un contenu utilisateur est ajouté, le poids ne dépend plus seulement de l'architecture mais aussi des données, ce qui est attendu et acceptable.

9.5 Conclusion

EduLib remplit sa double promesse. Côté fonctionnel : une bibliothèque collaborative complète, sécurisée et utilisable, conforme au CRUD complet exigé par le sujet (utilisateurs + entité métier + administration + votes / favoris / commentaires). Côté Green IT : un score EcoIndex A, une page d'accueil sous les 35 Ko, deux requêtes HTTP, zéro tracker, zéro cookie tiers, et une empreinte CO₂ inférieure d'un facteur 22 à 131 par rapport à un site comparable du marché.

La conclusion principale, en accord avec la citation de Saint-Exupéry retenue par le sujet, est que la sobriété numérique se construit surtout en retirant — frameworks, polices, trackers, animations, images décoratives — plutôt qu'en ajoutant des optimisations. La vraie performance Green IT vient des décisions de conception, pas des micro-optimisations de fin de projet.

Annexes

A. Variables d'environnement (.env.example)

```
DB_HOST=localhost DB_NAME=edulib DB_USER=root DB_PASS=votre_mot_de_passe
SITE_NAME=EduLib # Laisser vide pour désactiver les notifications email
MAIL_FROM=noreply@votre-domaine.fr
```

B. Catégories disponibles

Informatique · Mathématiques · Physique · Électronique · Gestion de projet · Langues · Économie · Autre.

C. Commandes de lancement

```
# 1. Cloner et configurer git clone https://github.com/mababou9143094800/Projet-
EduLib.git cd Projet-EduLib && cp .env.example .env # 2. Initialiser la base mysql -u
root -p edulib < sql/schema.sql # 3. Lancer le serveur de développement php -S
localhost:8000 # 4. Tests composer install && composer test
```

D. Structure résumée des fichiers PHP (≈ 70 fichiers)

```
Pages publiques : index.php · resources.php · resource-view.php
login.php · register.php · logout.php Pages connectées : dashboard.php · profile.php
add-resource.php · edit-resource.php · delete-resource.php Endpoints : vote.php
· favorite.php · comment.php SEO : sitemap.php · robots.txt Admin :
admin/index.php · admin/users.php · admin/resources.php admin/edit-
user.php · admin/delete-user.php admin/bulk-users.php · admin/bulk-
resources.php admin/includes/admin-nav.php Includes :
includes/config.php · includes/db.php · includes/auth.php
includes/functions.php · includes/header.php · includes/footer.php SQL :
sql/schema.sql · sql/migrate.sql Tests : tests/AuthTest.php ·
tests/FunctionsTest.php · tests/bootstrap.php
```